Plano de Gerenciamento de Configuração — CronLegível

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD) · Versão 1.0

Campo	Descrição
Projeto	CronLegível — app web que traduz expressões cron para português e mostra as próximas execuções
Gerente de Projeto / Product Owner	Desenvolvedor Líder (@Vzleopoldo)
Responsável de GCS-DevOps	Desenvolvedor Líder (@Vzleopoldo)
Equipe	1 Desenvolvedor Líder + 1 par de revisão (@FreitasArthur)
Repositório principal	https://github.com/Vzleopoldo/cronlegivel (GitHub, plano gratuito, público)
Regime de trabalho	Remoto e assíncrono, dedicação parcial (projeto acadêmico)

Histórico de Revisões

Data	Versão	Descrição	Autor
09/11/2026	1.0	Elaboração do plano a partir do PGC de referência da ONGConecta, adaptado ao CronLegível (app web estático, sem backend nem mobile, equipe de 2 pessoas).	Desenvolvedor Líder

Sumário

1. Introdução
2. Cultura DevOps e Papéis na Gerência de Configuração
3. Ferramentas, Ambientes e Infraestrutura
4. Identificação da Configuração
5. Integração Contínua (CI)
6. Entrega e Implantação Contínua (CD)
7. Infraestrutura como Código (IaC)
8. Controle de Configuração e Mudança
9. Baselines e Releases
10. Estimativa do Status de Configuração
11. Marcos
12. Treinamento e Recursos
13. Controle de Dependências de Terceiros
14. Quadro-Resumo do Pipeline CI/CD

> **Legenda de situação** usada nas tabelas: ✅ implantado · ➡️ SOON previsto (aula indicada).

---

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) do CronLegível ao longo do seu ciclo de vida, com ênfase em práticas DevOps: Integração Contínua (CI), Entrega Contínua (CD), automação de pipelines e uso exclusivo de ferramentas gratuitas.

O plano parte do PGC de referência da ONGConecta e o simplifica onde o projeto é menor:

1. **Só web estática.** Não há backend, banco de dados nem aplicativo mobile. O build é um conjunto de arquivos HTML/CSS/JS gerado pelo Vite.
2. **Equipe de 2 pessoas.** Os papéis do template são consolidados no Desenvolvedor Líder; o par de revisão garante que nenhuma mudança entre na `main` sem um segundo olhar.
3. **Sem orçamento.** Tudo roda no GitHub (repositório, Actions, Pages, Environments, Dependabot).

1.1 Finalidade

Estabelecer um padrão simples e verificável que garanta rastreabilidade, versionamento e controle de mudanças do CronLegível, apoiado por pipelines automatizados de lint, teste, build e implantação.

1.2 Escopo

Aplica-se a todo o conteúdo do repositório `Vzleopoldo/cronlegivel`: código-fonte, testes, configurações de build e lint, definições de pipeline, templates de colaboração e documentação. Não há entregáveis fora do repositório além do site publicado.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
PGC	Plano de Gerenciamento de Configuração (este documento)
CI	Integração Contínua (*Continuous Integration*)
CD	Entrega Contínua (*Continuous Delivery*)
CCM	Comitê de Controle de Mudanças
IC	Item de Configuração
Baseline	Conjunto de ICs que atingiu um estado comprovado de estabilidade (tag de versão)
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
GitHub Flow	Branching com `main` sempre implantável e branches de vida curta
SemVer	Versionamento Semântico (`MAJOR.MINOR.PATCH`)

Termo	Significado
Expressão cron	Agenda de 5 campos (minuto, hora, dia do mês, mês, dia da semana)

1.4 Referências

- PGC de referência da ONGConecta — *Plano de Gerenciamento de Configuração, Enfoque DevOps (CI/CD)*, v1.0.
- Repositório de referência: https://github.com/lzcamargo/ongconecta (atividades em `docs/aulas/`).
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Documentação oficial do GitHub (Actions, Pages, Environments, branch protection, Dependabot).
- README.md do CronLegível (como instalar, rodar e testar).

1.5 Visão Geral

A GCS é tratada como parte do fluxo de entrega, não como atividade manual em marcos formais: cada Pull Request revisado, testado e implantado gera evidência de rastreabilidade (histórico de PR, logs do pipeline, tags de versão e deploys registrados nos Environments).

---

2. Cultura DevOps e Papéis na Gerência de Configuração

Com 2 pessoas, os papéis do template são consolidados. A regra que não se consolida é a da revisão: **quem escreve a mudança não é quem a aprova.**

Papel (template)	Quem exerce	Responsabilidades
Gerente de Projeto / Product Owner	Desenvolvedor Líder	Prioriza as Issues e decide o que entra em cada release.
Engenheiro DevOps / Release Engineer	Desenvolvedor Líder	Mantém pipelines, proteção da `main`, Environments e versionamento.
Desenvolvedor	Desenvolvedor Líder	Implementa funcionalidades e testes.
Revisor (par de revisão)	@FreitasArthur	Revisa e aprova Pull Requests (é *code owner* de todo o repositório). Acesso de escrita.
QA / Automação de Testes	Desenvolvedor Líder	Mantém a suíte Vitest; o pipeline substitui QA manual.
Segurança (DevSecOps)	Desenvolvedor Líder	Acompanha Dependabot e *secret scanning*; garante que nenhum segredo seja versionado.
Aprovador de produção	Desenvolvedor Líder	Revisor obrigatório do Environment `production`.
CCM	Desenvolvedor Líder + par de revisão	Só para mudanças de alto risco (seção 8.2).
SRE / Operações	Desenvolvedor Líder	Verifica o site após cada deploy de produção.

2.1 Comunicação assíncrona

- Descrições de PR completas (contexto, o que mudou, como testar), seguindo o template de PR, para que a revisão não dependa de reunião.

- Discussões técnicas ficam no próprio PR ou na Issue, para manter o histórico no repositório.
- Prazo de resposta a um PR: **até 3 dias úteis**.

---

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta	Finalidade	Situação
Controle de versão	Git + GitHub (plano gratuito)	Código, pipelines como código e documentação	✅
Runtime e dependências	Node 22 (`.nvmrc`) + npm (`package-lock.json`)	Versões exatas e reprodutíveis	✅
Build	Vite	Gera o site estático em `dist/`	✅
Testes e cobertura	Vitest + `@vitest/coverage-v8`	Testes unitários e quality gate de cobertura	✅
Análise estática	ESLint	Lint bloqueante	✅
Orquestração de CI/CD	GitHub Actions	Automação de lint, testes, build e deploy	➡️ SOON Aula 2/3
Hospedagem	GitHub Pages (branch `gh-pages`)	Staging e produção sem conta externa (mesma decisão da ADR 0001 da ONGConecta)	➡️ SOON Aula 3
Aprovação de deploy	GitHub Environments	Gate manual para produção	✅
Gestão de segredos	GitHub Secrets	Credenciais do pipeline (hoje o app não precisa de nenhuma)	✅
Dependências / SCA	Dependabot + `npm audit`	Vulnerabilidades e PRs de atualização	➡️ SOON Aula 2/4
Rastreamento de tarefas	GitHub Issues (templates `funcionalidade` e `defeito`)	Vincular PRs a itens de trabalho	✅

Monitoramento com Sentry/UptimeRobot, previsto no PGC de referência, **não é adotado**: o CronLegível roda inteiramente no navegador e o GitHub Pages cuida da disponibilidade (ver 10.1).

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção	Configuração no GitHub	Situação
Desenvolvimento	Máquina do desenvolvedor (`npm run dev`)	—	—	✅
`staging`	Validar o build publicado antes de produção (`/cronlegivel/staging/`)	Merge na `main`	Environment `staging` : só branches protegidas	✅ criado · ➡️ SOON deploy na Aula 3

Ambiente	Finalidade	Gatilho de promoção	Configuração no GitHub	Situação
`production`	Site público (`/cronlegivel/`)	Aprovação manual do Desenvolvedor Líder após staging	Environment `production` : revisor obrigatório @Vzleopoldo, *prevent self-review* desligado, só branches protegidas	✅ criado · ➡️ SOON deploy na Aula 3

O build usa caminhos relativos (`base: './'` em `vite.config.js`), então **o mesmo artefato** validado em staging é promovido para produção, sem recompilar.

3.3 Estratégia de Branching e Versionamento

GitHub Flow:

- `main` — sempre implantável. **Protegida** (regra clássica):
  - Pull Request obrigatório, com **1 aprovação**;
  - aprovações descartadas quando chegam novos commits (*dismiss stale reviews*);
  - **histórico linear**;
  - **sem bypass**, nem para administradores;
  - *force push* e exclusão bloqueados;
  - checks obrigatórios `ci-ok` e `titulo-pr` (➡️ SOON Aula 2).
- `feat/<issue>-<descricao>` , `fix/<issue>-<descricao>` , `docs/<descricao>` , `chore/<descricao>` — branches de vida curta, apagadas automaticamente após o merge.
- `hotfix/<descricao>` — correção urgente; segue o mesmo fluxo de PR, com prioridade de revisão.

Merge: somente *squash merge*. Cada PR vira **um** commit na `main` , com o **título do PR** como mensagem e a descrição como corpo. Merge commit e rebase merge estão desabilitados.

Versionamento: SemVer + Conventional Commits (`feat` , `fix` , `docs` , `chore` , `refactor` , `test` , `ci`). Como o título do PR vira a mensagem do commit na `main` , o pipeline calcula a próxima versão a partir dele: `feat` → MINOR, `fix` → PATCH, `!` / `BREAKING CHANGE` → MAJOR.

---

4. Identificação da Configuração

4.1 Itens de Configuração

Item de configuração	Caminho
Código-fonte da aplicação	`src/cron.js` , `src/main.js` , `index.html` , `src/style.css`
Testes automatizados	`src/cron.test.js`
Configuração de build e testes	`vite.config.js` (inclui o limite de cobertura)
Configuração de lint	`eslint.config.js`

Item de configuração	Caminho
Manifesto e lockfile de dependências	`package.json`, `package-lock.json`
Versão do runtime	`.nvmrc`
Normalização de finais de linha	`.gitattributes` (`* text=auto eol=lf`)
Exclusões de versionamento	`.gitignore` (`node_modules/`, `dist/`, `coverage/`, `.env*`)
Variáveis de ambiente (somente nomes)	`.env.example`
Templates de colaboração	`.github/pull_request_template.md`, `.github/ISSUE_TEMPLATE/*.yml`
Responsáveis por revisão	`.github/CODEOWNERS`
Pipelines como código	`.github/workflows/*.yml` (SOON Aula 2/3)
Documentação	`README.md`, `docs/plano-gcs.md` (este plano), `docs/adr/` (quando houver decisões)

Não são itens de configuração e não são versionados: `node_modules/`, `dist/`, `coverage/` e qualquer `.env` com valores reais. **Segredos nunca entram no repositório**: ficam em GitHub Secrets. Se algum segredo for commitado por engano, ele é trocado na origem (o histórico do Git o preserva mesmo após apagar o arquivo).

As configurações do repositório no GitHub (proteção da `main`, regras de merge, Environments, colaboradores) também fazem parte da configuração do projeto. Elas estão descritas neste plano (seções 2, 3.2 e 3.3), e qualquer alteração nelas deve ser registrada aqui por PR.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch	`<tipo>/<issue>-<descricao-curta>`	`feat/12-nomes-de-meses`
Título de PR / commit na `main`	Conventional Commits: `<tipo>(<escopo opcional>): <descrição>`	`feat: suporte a nomes de meses (JAN-DEC)`
Tag de release	`vX.Y.Z`	`v1.1.0`
Arquivo de ADR	`docs/adr/NNNN-titulo-curto.md`	`docs/adr/0001-github-pages.md`

4.3 Estrutura do Repositório

```
.github/            templates de PR e Issue, CODEOWNERS, workflows (Aula 2+)
docs/               este plano e ADRs
src/                código-fonte e testes
index.html          página
vite.config.js      build e quality gate de cobertura
eslint.config.js    lint
package.json        scripts e dependências (+ package-lock.json)
```

5. Integração Contínua (CI)

🔜 **Aula 2.** Todo Pull Request para a `main` dispara o pipeline de CI. Nenhuma mudança é integrada sem pipeline verde.

5.1 Etapas do Pipeline de CI

Etapa	Comando
Checkout e dependências (com cache do npm)	`npm ci`
Análise estática	`npm run lint`
Testes unitários + cobertura (quality gate)	`npm run test:coverage`
Build	`npm run build`
Análise de dependências	`npm audit --audit-level=high` + Dependabot
Validação do título do PR (Conventional Commits)	workflow `pr-titulo.yml` → check `titulo-pr`
Check agregador	job `ci-ok` (só fica verde se nenhum outro job falhou)

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige **1 aprovação** de outra pessoa (o par de revisão, que é *code owner*).
- Novos commits no PR descartam aprovações anteriores.
- PRs pequenos e descritivos, preenchendo o template.
- Merge somente com `ci-ok` e `titulo-pr` verdes (🔜 Aula 2).
- Merge por *squash*, com o título do PR como mensagem.

5.3 Critérios de Qualidade (Quality Gate)

Critério	Limite	Onde é verificado
Cobertura de linhas de `src/` (exceto `src/main.js`, que só liga o DOM)	**≥ 80%** (acima dos 60% do PGC de referência)	`thresholds` em `vite.config.js`
Erros de lint	0	`npm run lint`
Testes	100% passando	`npm test`
Vulnerabilidades altas/críticas em dependências	0	`npm audit` / Dependabot
Build	sem erro	`npm run build`

6. Entrega e Implantação Contínua (CD)

🔜 **Aula 3.** Depois do merge na `main`, o pipeline de CD gera **um** artefato e o promove de staging para produção. Não há trilha mobile.

6.1 Etapas do Pipeline de CD — Web

1. Build único do site (`npm run build`).
2. Deploy automático em **staging** (GitHub Pages, `/cronlegivel/staging/`), no Environment `staging` .
3. Verificação de que o site de staging responde.
4. **Aprovação manual** do Desenvolvedor Líder no Environment `production` .
5. Deploy do **mesmo artefato** em produção (`/cronlegivel/`).
6. Verificação pós-deploy.
7. Criação da tag `vX.Y.Z` e das notas de versão (seção 9).

6.2 Etapas do Pipeline de CD — Mobile

Não se aplica: o CronLegível não tem aplicativo mobile.

6.3 Estratégias de Implantação

Estratégia	Uso no projeto
Promoção de artefato (staging → produção)	Padrão: o build validado em staging é o mesmo publicado em produção.
Gate de aprovação manual	Environment `production` com revisor obrigatório.
Feature flags	Não adotadas por ora: o app é pequeno e o rollback é rápido (6.4).

6.4 Rollback e Recuperação

Para voltar à última versão boa, republica-se a tag anterior (`vX.Y.Z`) em produção pelo pipeline (workflow de rollback, SOON Aula 3). A correção definitiva segue o fluxo normal de PR. Como não há banco de dados nem estado no servidor, o rollback não exige migração de dados.

---

7. Infraestrutura como Código (IaC)

Não há infraestrutura própria: o site é estático e hospedado no GitHub Pages. Ferramentas como Terraform/Ansible não se aplicam. O princípio de "tudo versionado" é mantido:

- pipelines e deploy ficam em `.github/workflows/` e mudam somente por PR;
- configurações de build (`vite.config.js`) e lint (`eslint.config.js`) são versionadas;
- as configurações do repositório no GitHub estão documentadas neste plano (4.1).

---

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Fluxo padrão para funcionalidades e correções:

Issue (template `funcionalidade` ou `defeito`) → **branch** → **Pull Request** (template, título Conventional Commits, `Closes #N`) → **CI verde** → **aprovação do par** → **squash merge** → **CD** (staging → aprovação → produção).

Esse fluxo já é o registro formal e rastreável da mudança. Não são usados formulários adicionais.

8.2 Comitê de Controle de Mudança (CCM)

Composto pelo Desenvolvedor Líder e pelo par de revisão. É acionado apenas para mudanças de alto risco:

- alterações nas regras de proteção da `main`, nas regras de merge ou nos Environments;
- alterações nos pipelines de CD ou no processo de rollback;
- mudanças de comportamento da tradução ou do cálculo de próximas execuções que alterem resultados já exibidos aos usuários;
- exceções ao fluxo padrão.

A decisão do CCM é registrada no PR (comentário de aprovação justificado) e, quando for uma decisão de arquitetura, em um ADR em `docs/adr/`. O template de Issue de mudança de alto risco será adicionado na Aula 4 ().

---

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada deploy bem-sucedido em produção gera uma **baseline**: a tag SemVer `vX.Y.Z` no commit publicado. A versão é calculada pelo pipeline a partir dos títulos dos PRs desde a última tag ( Aula 3). A baseline inicial é o commit de versão `1.0.0` do `package.json`.

9.2 Notas de Versão (Release Notes)

Geradas automaticamente pelo pipeline a partir dos títulos dos PRs (Conventional Commits) desde a última tag e publicadas como **GitHub Release** junto à tag.

---

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O CronLegível não tem servidor: todo o processamento acontece no navegador do usuário. Por isso, o monitoramento se limita a:

- verificação automática de que o site responde após cada deploy (seção 6.1);
- aba **Deployments** do GitHub (histórico de staging e produção por Environment);
- Issues de `defeito` abertas por usuários.

10.2 Métricas acompanhadas

Métrica	Descrição	Fonte
Deployment Frequency	Deploys em produção por período	Histórico do Environment `production`
Change Failure Rate	% de deploys em produção que precisaram de rollback ou hotfix	Execuções de rollback e PRs `fix` pós-release

Lead Time for Changes e *MTTR* poderão ser adotadas quando houver histórico suficiente.

10.3 Relatórios e Auditorias

O histórico do GitHub (PRs com revisões, execuções do Actions, deploys dos Environments, tags e Releases) é o insumo de auditoria de GCS. Não são produzidos relatórios manuais.

---

11. Marcos

Marco	Situação
Itens de configuração básicos, par de revisão, regras de merge, Environments e proteção da `main`	✅ Aula 1
Pipeline de CI com quality gate e checks obrigatórios	➡️ SOON Aula 2
CD para staging, aprovação, produção, baseline e rollback	➡️ SOON Aula 3
Governança: dependências, mudanças de alto risco e rastreabilidade	➡️ SOON Aula 4
Primeira release totalmente automatizada (do PR à produção, com intervenção manual apenas no gate)	➡️ SOON Aula 3

---

12. Treinamento e Recursos

- O README é o guia de onboarding: como instalar, rodar, testar e contribuir.
- Decisões técnicas relevantes são registradas como ADRs curtos em `docs/adr/`.
- Revisões de PR são usadas como troca de conhecimento: comentários explicam o porquê, não só aprovam ou rejeitam.
- O processo de release e rollback será documentado quando o pipeline de CD estiver implantado.

---

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Não há subcontratados nem serviços de terceiros em produção além do próprio GitHub. As dependências de terceiros são **apenas de desenvolvimento** (Vite, Vitest, ESLint e plugins): o site publicado não carrega bibliotecas externas.

- Versões exatas fixadas no `package-lock.json` e instaladas com `npm ci`.
- `npm audit` no pipeline e Dependabot habilitado (➡ SOON Aula 2/4).
- Atualizações de dependências seguem o mesmo fluxo de PR + revisão + pipeline.

---

14. Quadro-Resumo do Pipeline CI/CD

Etapa	Descrição	Gatilho
CI	Lint, testes com cobertura ≥ 80%, build, `npm audit`, título do PR	Abertura/atualização de PR
Merge	Squash na `main`, título do PR como mensagem	1 aprovação do par + `ci-ok` e `titulo-pr` verdes
CD — Staging	Deploy do build no GitHub Pages (`/staging/`)	Merge na `main`
Gate de release	Aprovação manual do Desenvolvedor Líder	Sucesso em staging
CD — Produção	Deploy do mesmo artefato, verificação, tag `vX.Y.Z` e Release	Aprovação do gate
Rollback	Republica a tag anterior em produção	Manual, em caso de falha